RAG-BASED CUSTOMER SUPPORT ASSISTANT

with LangGraph & Human-in-the-Loop (HITL)

DELIVERABLE 3

TECHNICAL DOCUMENTATION

Internship Project Document

Domain: AI / Machine Learning

Technology Stack: Python · FastAPI · LangGraph · ChromaDB · Groq LLM

1. Introduction

1.1 What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is a two-stage AI architecture designed to ground Large Language Model (LLM) outputs in factual, domain-specific information. Unlike a standalone LLM that generates answers purely from patterns learned during training, a RAG system first retrieves relevant documents or passages from an external knowledge base, then provides those passages as context to the LLM for answer generation.

The retrieval stage uses semantic search — converting both documents and queries into dense vector representations (embeddings) and finding the most similar documents using distance metrics such as cosine similarity. The generation stage uses the retrieved content as a grounding context, instructing the LLM to answer only based on that content, not its training memory.

This two-stage approach combines the natural language understanding and fluency of LLMs with the precision and freshness of a curated knowledge base, yielding a system that is both accurate and easy to update.

1.2 Why is RAG Needed?

RAG solves three fundamental problems that arise when deploying LLMs in production environments:

- Hallucination Prevention: LLMs frequently generate confident but incorrect answers when asked about specific domain knowledge (product specs, policies, prices). By constraining the model to answer only from retrieved context, RAG dramatically reduces hallucination rates.
- Knowledge Currency: LLM training data has a cutoff date. RAG allows the knowledge base to be updated independently of the model — simply re-ingesting updated PDFs is sufficient to keep the system current.
- Cost Efficiency: Fine-tuning an LLM for every domain or every policy update is prohibitively expensive. RAG achieves domain adaptation through retrieval, requiring no model retraining.

1.3 Use Case Overview

This system implements a RAG-powered customer support assistant for a business with a PDF-based knowledge base (product manuals, policy documents, FAQs). The assistant:

- Receives natural language queries from customers via a web chat interface
- Retrieves relevant sections from the uploaded PDF knowledge base
- Generates accurate, grounded answers using Groq (LLM)
- Detects query intent and routes accordingly: simple greetings, knowledge queries, or sensitive escalations
- Escalates sensitive or ambiguous queries to human agents with full ticket tracking
- Provides a built-in agent dashboard for human agents to respond to escalated tickets

2. System Architecture Explanation

2.1 Detailed Explanation of the HLD

The system is organized as four distinct horizontal layers, each with clearly defined responsibilities and interfaces.

Layer 1 — Browser Frontend (HTML/CSS/JavaScript)

The frontend is a single-page application built with vanilla HTML, CSS, and JavaScript — no framework dependencies. It consists of two functional areas: a chat window for user interaction, and a sidebar containing the PDF uploader, system status panel, and HITL agent dashboard. Communication with the backend occurs exclusively via REST API calls using the browser's native fetch() API. The UI updates in real time after each API response, appending message bubbles with metadata badges showing intent, confidence, and escalation status.

Layer 2 — FastAPI Backend

FastAPI serves as the API gateway between the browser and the intelligence layer. It handles file uploads (/ingest), chat requests (/chat), and HITL ticket management (/hitl/*). FastAPI also serves the frontend's static files, making the entire application available from a single process on port 8000. All components — EmbeddingModel, VectorStore, LLMHandler, HITLManager, and RAGWorkflow — are instantiated once at startup and reused across requests.

Layer 3 — Intelligence Layer (LangGraph + RAG Pipeline)

This is the core of the system. It consists of six interconnected processing modules: DocumentProcessor, EmbeddingModel, VectorStore, Retriever, LLMHandler, and the LangGraph workflow. The workflow is a compiled directed state graph that orchestrates the other modules. Each user query triggers a complete traversal of the graph from the intent_router entry point to one of three possible terminal nodes (simple_reply, generate_answer, or escalate_hitl).

Layer 4 — Persistence Layer

Two persistence mechanisms are used. ChromaDB stores document embeddings on disk in the chroma_db/ directory using an HNSW (Hierarchical Navigable Small World) index for fast approximate nearest-neighbour search.

2.2 Component Interactions

The following describes how components interact during a typical knowledge query:

1. User submits query in the browser → JavaScript calls POST /chat
2. FastAPI receives the ChatRequest → calls RAGWorkflow.run(query, session_id)
3. intent_router node calls Router.detect_intent(query, confidence=1.0)
4. retrieve node calls EmbeddingModel.embed_query(query) to get a 384-dim vector
5. retrieve node calls VectorStore.query(vector, top_k=5) → gets List[(Document, float)]
6. grade_documents node calls Router.detect_intent(query, real_confidence) for final routing

7. generate_answer node calls LLMHandler.generate_answer(query, context_docs)
8. LLMHandler calls Groq API with system prompt + context + query
9. Groq returns answer → stored in GraphState → returned to FastAPI → serialized to JSON → sent to browser

3. Design Decisions

3.1 Chunk Size Choice — 500 Characters

The chunk size of 500 characters was chosen based on the following analysis:

- Embedding model context: all-MiniLM-L6-v2 has a 256 token (approx. 1000 character) input limit. At 500 characters, chunks are well within the limit with headroom for tokenization overhead.
- Semantic coherence: 500 characters is large enough to contain 2-4 complete sentences, preserving enough context for the embedding to be meaningful.
- Retrieval precision: shorter chunks produce more specific embeddings with less noise, improving retrieval accuracy for focused questions.
- A chunk overlap of 50 characters (10% of chunk size) ensures sentences at chunk boundaries are captured in adjacent chunks, preventing information fragmentation.

3.2 Embedding Strategy

The choice of all-MiniLM-L6-v2 as the embedding model was driven by the following criteria:

Criterion	Value / Explanation
Model size	80MB — fits in RAM on any modern machine
Embedding dimension	384 — compact vector, fast to index and query
Inference speed	~5ms per sentence on CPU — acceptable for real-time use
Training data	Trained on 1B+ sentence pairs — strong semantic similarity
API dependency	None — runs 100% locally
Alternatives considered	OpenAI text-embedding-3-small (API cost), BERT-base (larger, slower), GTE-large (higher accuracy but 3x size)

3.3 Retrieval Approach

Cosine similarity was chosen over Euclidean distance because text embeddings encode semantic meaning as directions, not magnitudes. Two semantically similar sentences will have high cosine similarity (small angle) regardless of their absolute vector magnitudes. The retrieval pipeline:

10. Embeds the user query with the same model used for document embedding (critical: must use identical embedding model)
11. Queries ChromaDB for top-5 results by cosine distance
12. Filters results with distance > 0.35 (empirically calibrated for MiniLM)

13. Computes confidence as 1 - mean_distance for routing decisions

3.4 Prompt Design Logic

The system prompt for Claude was designed with three principles:

- Context restriction: 'Answer ONLY using information from the provided context' — this is the primary hallucination prevention mechanism
- Graceful degradation: 'If context does not contain enough information, say you cannot find it' — users get an honest response rather than a fabricated one
- Tone guidance: 'Acknowledge user feelings first if they seem upset' — improves customer satisfaction in support scenarios

The user message template follows the format: Context: [numbered chunks] → Question: [query] → Answer: — this clear demarcation helps the model correctly distinguish context from question.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph's StateGraph class was chosen to orchestrate the pipeline for reasons beyond simply connecting steps. The key advantages over a sequential chain or simple function calls are:

- Explicit branching: conditional edges in LangGraph make the routing logic visible in the graph structure itself, not buried in application code
- Typed state contract: TypedDict ensures every node receives and produces a consistent state shape, catching bugs early
- Testability: individual nodes can be tested in isolation by passing mock GraphState dictionaries
- Extensibility: adding a memory node, a re-ranking node, or a cache-check node requires only defining the node method and adding an edge — no refactoring of existing nodes
- Debuggability: the state after each node can be logged or inspected, providing a complete audit trail of each query's processing path

4.2 Node Responsibilities in Detail

Node	Primary Role	Key Logic
intent_router	Entry point classification	Keyword matching on lowercased query. Sets intent and reason. No external calls.
retrieve	Vector similarity search	Embeds query → ChromaDB query → filters by threshold → computes confidence.
grade_documents	Post-retrieval re-classification	Re-evaluates intent with real confidence score. May promote KNOWLEDGE to ESCALATION.
generate_answer	LLM answer generation	Builds numbered context block → calls Claude API → returns grounded answer string.
simple_reply	Bypass for trivial intents	Returns static greeting or farewell string. Zero latency. No external calls.
escalate_hitl	Human escalation	Creates persisted ticket with UUID → generates user acknowledgement message.

4.3 State Transitions

The GraphState TypedDict flows through nodes as an immutable dictionary. Each node uses Python's dict spread pattern ({**state, key: new_value}) to produce a new state, ensuring no node mutates shared state. The initial state contains only query and session_id — all other fields start empty or at default values. The final state after graph completion contains the complete answer field plus metadata about how the query was processed.

5. Conditional Logic

5.1 Intent Detection

Intent detection occurs at two points in the graph — before retrieval (in intent_router) and after retrieval (in grade_documents). This two-pass design ensures:

- Low-latency shortcircuiting: greetings and obvious escalations never trigger expensive embedding and retrieval
- Accurate post-retrieval escalation: queries that seem answerable by keywords but actually have no relevant content in the knowledge base are still escalated

The detection priority order is critical. It is evaluated strictly top-to-bottom with the first match winning:

```
Priority 1: Greeting keywords      → GREETING   (bypass retrieval)
Priority 2: Farewell keywords      → FAREWELL   (bypass retrieval)
Priority 3: Escalation keywords    → ESCALATION (bypass retrieval)
Priority 4: Low confidence (post)  → ESCALATION (after retrieval)
Priority 5: Default                → KNOWLEDGE  (proceed to generate)
```

5.2 Routing Decisions

The routing functions _route_after_intent and _route_after_grade return string keys that LangGraph maps to the next node. This design cleanly separates routing logic from node logic:

Routing Point	Condition	Next Node
After intent_router	intent = GREETING or FAREWELL	simple_reply
After intent_router	intent = ESCALATION	escalate_hitl
After intent_router	intent = KNOWLEDGE	retrieve
After grade_documents	intent = ESCALATION	escalate_hitl
After grade_documents	intent = KNOWLEDGE	generate_answer

6. HITL Implementation

6.1 Role of Human Intervention

The HITL mechanism exists to handle the cases where AI-generated responses would be inappropriate, risky, or simply unavailable. Human intervention is specifically targeted at three scenarios:

- Sensitive topics: refunds, legal claims, fraud, account cancellations — where an incorrect AI response could have financial or legal consequences
- Knowledge gaps: queries that the knowledge base genuinely cannot answer, where the right response is connecting the user to a knowledgeable person
- Emotionally escalated customers: users expressing frustration or urgency who need empathetic human interaction, not just a correct answer

The HITL system is designed as a supplement to the AI, not a fallback that signals AI failure. Escalation is presented to users as a premium service (connecting to a specialist), not as an error state.

6.2 Benefits

- Safety: prevents the AI from providing wrong guidance on sensitive financial, legal, or health topics
- Trust: users know a human is always available, increasing confidence in the overall system
- Audit trail: every escalation is logged with timestamp, reason, query, and human response
- Seamless UX: the user never leaves the chat window — the agent response appears in the same thread
- Reduced agent load: routine queries are handled automatically, agents only deal with cases requiring judgment

6.3 Limitations

- Asynchronous response: the agent must be online and actively monitoring the dashboard; there is no real-time notification system in the current implementation
- Single-node persistence: the JSON file approach does not support concurrent writes from multiple server instances
- No SLA tracking: response time targets are not enforced or measured
- No escalation categorization: all tickets are treated equally regardless of urgency level
- No agent authentication: the dashboard is not protected by login in the current scope

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs. Speed

The primary trade-off in retrieval is between accuracy (returning the most relevant chunks) and speed (returning results quickly).

Approach	Accuracy	Speed	Chosen?
top_k=5, threshold=0.35	Good for focused queries	~10ms query	Yes — balanced choice
top_k=20, no threshold	Higher recall, more noise	~30ms query	No — too much noise for LLM
Re-ranking with cross-encoder	Best accuracy	~200ms extra	Future enhancement
Exact search (brute force)	Perfect	Slow at 1M+ docs	No — not scalable

7.2 Chunk Size vs. Context Quality

Chunk Size	Advantage	Disadvantage
Very small (100 chars)	Highly specific embeddings	Loses sentence context, fragments meaning
Small (500 chars) — CHOSEN	Good balance of specificity and context	May miss multi-paragraph reasoning
Large (1000+ chars)	Captures full paragraphs and arguments	Embedding is diluted, retrieval less precise

7.3 Cost vs. Performance

Component	Cost-Optimized Choice	Performance-Optimized Alternative	Impact of Upgrading
LLM	Claude Haiku (~$0.25/1M input tokens)	Claude Opus (~$15/1M tokens)	Significantly better reasoning on complex queries
Embeddings	Local MiniLM (free)	OpenAI text-embedding-3-large (paid)	Higher accuracy on technical domain text
Vector DB	ChromaDB (local)	Pinecone or Weaviate (paid cloud)	Horizontal scaling, multi-tenant support
HITL storage	JSON file (free)	PostgreSQL with Redis pub/sub	Real-time push, concurrent writes, SLA tracking

8. Testing Strategy

8.1 Testing Approach

The system supports three levels of testing, each targeting a different layer of the stack:

Unit Tests (Individual Modules):

- DocumentProcessor: assert chunk count > 0, assert metadata has 'source' and 'page' keys, assert no chunk exceeds chunk_size characters
- EmbeddingModel: assert output dimension = 384, assert embed_texts returns same count as input
- VectorStore: test upsert then query returns correct document, test clear() resets count to 0
- Retriever: mock VectorStore to return known distances, assert confidence computation is correct
- Router: assert each keyword triggers correct intent, assert confidence threshold enforced
- HITLManager: test create → resolve lifecycle, assert status transitions correctly

Integration Tests (API Endpoints):

- POST /ingest with sample PDF: assert HTTP 200, assert stats.total_chunks > 0
- POST /chat with answerable query: assert answer is non-empty, intent='knowledge'
- POST /chat with greeting: assert intent='greeting', no retrieval logged
- POST /chat with escalation keyword: assert escalated=True, ticket_id is not null
- GET /hitl/tickets: assert returns list
- POST /hitl/respond with valid ticket_id: assert status becomes 'resolved'

8.2 Sample Test Queries

Query	Expected Intent	Expected Behavior	Pass Criteria
Hello there!	GREETING	Greeting returned, no retrieval	intent='greeting', confidence=1.0
What is the return policy?	KNOWLEDGE	RAG retrieves and answers	intent='knowledge', answer non-empty
I want a refund NOW	ESCALATION	Ticket created	escalated=True, ticket_id not null
Explain the warranty terms	KNOWLEDGE	RAG retrieves warranty section	confidence > 0.5, relevant answer
I need to speak to a manager	ESCALATION	Ticket created — keyword match	escalated=True, reason contains 'manager'
aksjdhaksjdh random text	ESCALATION	Low confidence → escalated	confidence < 0.5, escalated=True
Thanks, bye!	FAREWELL	Farewell message returned	intent='farewell', no ticket

9. Future Enhancements

9.1 Multi-Document Support

The current implementation ingests all PDFs into a single ChromaDB collection. Production enhancement would include:

- Per-document namespacing using collection names or metadata filtering
- Document version management — tracking which PDF version is currently active
- A document management UI for viewing, replacing, and deleting ingested documents
- Automatic re-ingestion when a document is updated, with delta-only processing for efficiency

9.2 Feedback Loop

User feedback can be used to improve retrieval quality over time:

- Thumbs up/down buttons on each bot response, logged to a feedback store
- Negative feedback triggers review by human agents for knowledge base updates
- Positive feedback can be used to build a golden dataset for retrieval fine-tuning
- Feedback analytics dashboard showing which query types have lowest satisfaction rates

9.3 Memory Integration

The current system is stateless — each query is processed independently with no memory of prior turns. Multi-turn conversation support would include:

- A memory node in the LangGraph workflow that maintains conversation history
- History compression: summarizing old turns to keep context within LLM token limits
- Session-based ChromaDB filtering: prioritizing documents relevant to the ongoing conversation topic
- User profile memory: remembering user preferences and prior issues across sessions

9.4 Deployment

Component	Local (Current)	Production Deployment
FastAPI server	uvicorn main:app --reload	Gunicorn + uvicorn workers behind Nginx
Vector database	ChromaDB local files	Pinecone (cloud) or Weaviate (self-hosted)
HITL storage	hitl_escalations.json	PostgreSQL with SQLAlchemy ORM
Agent notifications	Manual dashboard refresh	WebSocket push + email/SMS via Twilio
Container	No container	Docker + docker-compose.yml
CI/CD	None	GitHub Actions for automated testing and deployment
Monitoring	Console logs	Datadog / Grafana for metrics, Sentry for errors